A Machine-Verified Core for the Pentagonal Structure of Prime Residues, with a Reproducible Discipline for AI-Assisted Mathematics

Christopher Brock
with proofs generated by Aristotle (Harmonic) and independently verified by AXLE (Axiom)

August 1, 2026

Abstract

We report a machine-verified core of the *Brockian* program on the pentagonal, golden-ratio structure of prime residues (the "Curved Number Line"). The core comprises 190 Lean 4 theorems (186 unconditional, 4 conditional) across 22 modules, checked against Mathlib and, crucially, *independently re-verified* by a second, cloud-hosted Lean engine. The mathematical highlights are: an exact admissibility law counting principal residues of prime k-tuples ($q-2$ admissible starts modulo a prime q, specialising to the twin constraint mod 3 and the Brockian case mod 5); a rigidity theorem showing that the golden value $\varphi - 1$ lies in the adjacency spectrum of the cycle graph C_p for *exactly one* prime, $p = 5$; the identity that the algebraic connectivity of the pentagon is golden, $\lambda_2(C_5) = 2 - 1/\varphi$; a faithful realisation of the dihedral group D_5 inside $\mathrm{Aut}(C_5)$; an exact local-covariance kernel for the Goldbach residual; and a "silver" spectral alphabet governing a sieve on prime constellations. Equally, we contribute the *method*: an audit ledger of named failure modes, a four-tier register (PROVED/COMPUTATION/CONDITIONAL/CONJECTURE), a *triple-verification* gate (local build + axiom check + independent re-check), and a machine-generated registry that makes it structurally impossible to label a result verified that the build does not earn. We are deliberate about what is *not* proved: the Riemann-Hypothesis attack is formalised only as an open schema, and its status is recorded as such. All artefacts, statements, axioms, and verdicts are public and reproducible.

1 Introduction

The "Curved Number Line" is a family of observations organising the integers on a golden spiral whose fifth-power symmetry sends prime residues into a small number of rays and constrains how prime constellations may sit on them. Over several years these observations accreted into a large body of exploratory mathematics and, more recently, into hundreds of machine-generated Lean 4 proof attempts. Exploratory AI-assisted mathematics of this kind faces a specific credibility problem: a proof assistant guarantees a proof, never its *statement*, and a generative prover will happily return a plausible-looking theorem whose hypotheses are vacuous, whose operator is secretly the zero map, or whose "proof" is a disguised `sorry`. Left unaudited, such a corpus is indistinguishable from numerology.

This paper does two things. First (§2) it describes the discipline we used to separate the real from the decorative: an audit ledger that names each failure mode as it is encountered, a small

register vocabulary applied to every declaration, and a *triple-verification* gate in which a result is called PROVED only if it (i) compiles, (ii) depends on no axioms beyond the three Mathlib admits by default (`propext`, `Classical.choice`, `Quot.sound`) and uses no compiler-trusting `native_decide`, and (iii) is independently re-verified by a *second*, unrelated Lean engine. Second (§§3–6) it presents the mathematical core that survived this discipline: 190 theorems whose exact statements, axiom footprints, and independent verdicts are published as a machine-generated registry (Table 1).

We are equally explicit about the program's horizon (§7). The Brockian attack on the Riemann Hypothesis, via a putative Hilbert–Pólya operator on the prime-Gaussian potential, is present here only as a *formalised schema*: its gates are stated, its one genuinely analytic obligation is reduced to a named classical criterion, and its central hypothesis is shown to be exactly of open-problem strength. Nothing in this paper should be read as progress *on* the Riemann Hypothesis; the contribution is a verified substrate and an honest accounting.

Contributions.

1. A verified core of 186 unconditional and 4 conditional Lean 4 theorems on the pentagonal/golden structure of prime residues, each independently re-checked (Table 1).
2. Two rigidity results singling out $p = 5$: $\varphi - 1 \in \mathrm{spec}(C_p) \iff p = 5$, and $\lambda_2(C_5) = 2 - 1/\varphi$.
3. An exact finite admissibility law $q - 2$ for prime constellations, with its analytic companion (positive, summable singular series).
4. A reproducible discipline — ledger, registers, triple verification, generated registry — offered as a template for honest AI-assisted mathematics, together with a catalogue of the failure modes it is designed to catch.

2 The verification discipline

2.1 Registers

Every declaration in the core carries exactly one *register*, and the register is *derived* from the compiled artefact, never hand-asserted:

proved sorry-free; `#print axioms` $\subseteq$ {`propext`, `Classical.choice`, `Quot.sound`}; no `native_decide` (which trusts the compiler and adds `Lean.ofReduceBool`); and independently re-verified (§2.2). Kernel-checked `decide` on finite goals is admitted here, as it adds no axioms.

computation finite checks that rely on `native_decide`; recorded as computations, never promoted to PROVED.

conditional depends on a named hypothesis, tagged with its *rung*: *classical* (a true theorem not yet in Mathlib), *literature* (a published result used as a hypothesis with citation), or *open* (an open-problem-strength assumption — admissible only as a schema, never as evidence).

conjecture a named `Prop` container, never a theorem.

2.2 Triple verification

A PROVED label requires three independent checks to agree: (1) a local `lake build` on the pinned toolchain; (2) a local axiom audit via `#print axioms`; and (3) an independent re-check by *AXLE* [3], a cloud-hosted Lean 4 engine that recompiles each proof against Mathlib at a named environment

(`lean-4.32.0`) and, via its `verify_proof` tool, checks that the proof discharges the intended statement rather than a drifted one. Because generative provers can self-report success while smuggling a custom axiom, no engine is trusted on its own word: the three gates run on every proof regardless of its provenance. A subtle but important point: a `sorry` raises a *warning*, not an error, so a naive "did it compile" check passes it; our gate treats any sorry/admit warning, and any `sorry`-derived axiom, as a hard failure.

2.3 The registry as single source of truth

The three checks feed a machine-generated registry (`registry/theorems.json`), one entry per declaration recording its statement, module, axiom set, independent verdict, register, and provenance. The register is *computed* from these facts, so the registry cannot claim more than the build earns; the table in this paper (Table 1) and the project's public results page are both generated from it. Declarations rejected during the audit are recorded, with their named failure mode, in a companion exclusion list — dropping decorative theorems silently would defeat the purpose.

2.4 Named failure modes

The audit accumulated a catalogue of concrete ways a machine-generated "theorem" can be hollow; each was met at least once in the corpus and is now screened for. Representative entries: *vacuous bounded-operator hypotheses* (a bounded operator cannot have the unbounded spectrum a Hilbert–Pólya operator needs, so the conditional holds vacuously); $\mathbb{R}$-*mod collapse* (a phase written `x % (2*π)` is identically 0 in a field, trivialising every angular claim); *Nat-division exponents* (`x^(1/2)` with a natural-number literal is $x^0 = 1$); *degenerate witnesses* (an existential identity satisfied by the zero object); *definitional theater* (a term "proved" equal to the definition it unfolds); and *overtitling* (a three-line lemma wearing a famous theorem's name). These are not hypothetical: the discipline exists because each occurred.

3 Golden algebra and the ray ring

Write $\varphi = (1 + \sqrt{5})/2$ for the golden ratio (Mathlib's `Real.goldenRatio`). The foundational module verifies the golden identities and the arithmetic of *rays*. Residues modulo 5 partition the nonzero classes; the reduction $\mathbb{N} \to \mathbb{Z}/5\mathbb{Z}$ is a ring homomorphism (`ray_add`, `ray_mul`), and a value lands on the zero ray iff it is a multiple of 5 (`ray_zero_iff_dvd`). The key non-elementary fact is imported from Mathlib's Dirichlet theorem:

Theorem 3.1 (`each_ray_has_infinitely_many_primes`, PROVED)**.** *For each unit $i \in \mathbb{Z}/5\mathbb{Z}$, the set $\{p \text{ prime} : p \equiv i \pmod 5\}$ is infinite.*

Alongside sit $\varphi^2 = \varphi + 1$ (`phi_sq`), $\cos(\pi/5) = \varphi/2$ (`cos_pi_div_five_eq_phi_div_two`), Binet's formula (`binet_formula`), and $5 \mid F_{5k}$ (`fib_five_dvd`). These are the substrate the later modules build on.

4 The admissibility law and the transition kernel

For a prime modulus q and a gap g, call a residue $a \in \mathbb{Z}/q\mathbb{Z}$ an *admissible start* if neither a nor $a + g$ is $\equiv 0$, i.e. $a \notin \{0, -g\}$.

Theorem 4.1 (`universal_admissibility_count`, PROVED)**.** *For any modulus $q \geq 1$ and gap $g \neq 0$ in $\mathbb{Z}/q\mathbb{Z}$, the number of admissible starts is exactly $q - 2$.*

Corollary 4.2 (`admissibility_count_three`, `admissibility_count_five`)**.** *Modulo 3 there is exactly 1 admissible start (the twin-prime constraint); modulo 5 there are exactly 3 (the Brockian case).*

This finite law is repackaged as a transition kernel on the active residues. Defining `kernel q g` to count admissible starts of each label stepping to the next, the double sum over the kernel equals the admissible-start count:

Theorem 4.3 (`totalSum_eq`, PROVED)**.** *For $q \geq 1$ and any gap g, $\sum_i \sum_j \mathit{kernel}\, q\, g\, i\, j = |\mathit{admissibleStarts}\, q\, g|$, and this equals $q - 2$ for $g \neq 0$.*

The kernel specialises to the constellation classification: the twin, cousin, sexy, and quadruplet configurations pin (or free) residues exactly as recorded by `twin_pins_mod_three`, `cousin_pins_mod_three`, `sexy_free_mod_three`, `quadruplet_pins_mod_five`, and the forbidden twin transition (`forbidden_transition`). On the analytic side, the Hardy–Littlewood local factor built from the same invariant ν_p is positive and its finite product is positive under admissibility (`local_factor_pos`, `singular_series_finite_pos`, `local_factor_asymptotic`); positivity of the full singular series is recorded as CONDITIONAL on convergence to a positive limit — a classical analytic fact stated as an explicit hypothesis rather than assumed as an axiom:

Theorem 4.4 (`singular_series_pos`, CONDITIONAL (rung: classical))**.** *If the finite singular-series products converge to some $S > 0$, then the singular series of G is positive.*

5 Spectral rigidity of the pentagon

Why five? Two verified rigidity theorems answer this in the language of the cycle graph. Model the adjacency spectrum of C_n concretely as `cycleSpectrum` $n = \{\, \mu : \exists k,\ \mu = 2\cos(2\pi k/n) \,\}$ (its circulant eigenvalues).

Theorem 5.1 (`golden_unique_to_five`, PROVED)**.** *For every prime p, $\varphi - 1 \in \mathit{cycleSpectrum}\, p \iff p = 5$.*

The forward direction is the content: from $\varphi - 1 = 2\cos(2\pi k/p) = 2\cos(2\pi/5)$ one deduces $5k = p(5m \pm 1)$ for some integer m, hence $5 \mid p$, hence $p = 5$. Thus among all primes, exactly one pentagon "sings" the golden ratio. The companion `neg_golden_in_C5_spectrum` records $-\varphi = 2\cos(4\pi/5) \in \mathrm{spec}(C_5)$.

The second rigidity is metric. The Laplacian of the 2-regular graph C_5 has eigenvalues $2 - 2\cos(2\pi k/5)$; its second-smallest (the algebraic connectivity) is golden:

Theorem 5.2 (`pentagon_lambda2_phi`, PROVED)**.** *$2 - 1/\varphi$ is a Laplacian eigenvalue of C_5, is positive, and is at most the other nonzero eigenvalue $2 - 2\cos(4\pi/5)$; hence $\lambda_2(C_5) = 2 - 1/\varphi = 3 - \varphi$, with ratio $\lambda_2/2 = 1 - 1/(2\varphi)$.*

Finally, the symmetry group. Mathlib 4.32 provides C_n but no graph-automorphism-group API, so we build the action by hand. The rotation $x \mapsto x + a$ and reflection $x \mapsto -b - x$ are verified to preserve adjacency (`rot_map_adj`, `refl_map_adj`), assembling into a group homomorphism $D_5 \to \mathrm{Aut}(C_5)$ that is injective:

Theorem 5.3 (`dihedral_action_faithful`, PROVED)**.** *The dihedral group D_5 acts faithfully on C_5 by graph automorphisms; consequently $10 \le |\mathrm{Aut}(C_5)|$ (`ten_le_card_aut`).*

Remark 5.4. The reverse bound $|\mathrm{Aut}(C_5)| \le 10$ — which would upgrade this to the full isomorphism $\mathrm{Aut}(C_5) \cong D_5$ — is *not* proved: Mathlib 4.32 offers no automorphism enumeration for cycle graphs, and we decline to assert what we cannot verify. The open direction is recorded as such.

6 A Goldbach comb and a silver sieve

Two further modules verify exact structure without claiming the hard theorems behind them.

Goldbach local covariance. For a prime p, the number of ordered pairs of nonzero residues summing to c is $g_p(c) = p - 2 + [c = 0]$ (`gCount_eq`), which centres to the spike $[c = 0] - 1/p$ (`gCount_centered`). The autocorrelation of the resulting weight is exact:

Theorem 6.1 (`local_covariance`, PROVED)**.** *For prime p and shift h, $\frac{1}{p}\sum_c g_p(c)\, g_p(c+h) = \big((p-1)^2/p\big)^2 K_p(h)$, with K_p lifting when $p \mid h$ and dipping otherwise.*

The passage from this local kernel to the global Goldbach residual is stated only as a named conjecture (§7); it is not claimed.

The silver alphabet. A sieve on prime triples yields a 3×3 Hamiltonian whose spectrum is the "silver" set: eigenvalues $2-\sqrt{2}$, 2, $2+\sqrt{2}$, with trace 6 and determinant 4 (`H3_ground`, `H3_middle`, `H3_top`, `H3_trace`, `H3_det`), and a finite rigidity of the gap set (`silver_gap_rigidity_finite`). Companion facts fix the twin admissibility count and a phase–depth torus of minimal period 25 (`tau_minimal_period`, `tau_injective_on_period`).

7 What is *not* proved: the open program

The value of the discipline is as much in its refusals as its theorems.

- **Riemann Hypothesis.** The Hilbert–Pólya attack is a *schema*. The intended operator is unbounded and densely defined (a bounded operator has compact spectrum and cannot carry the zeros' unbounded ordinates — a constraint we learned from a failed formalisation). The prime-Gaussian potential is a bounded self-adjoint multiplication operator; essential self-adjointness reduces to Weyl's limit-point criterion, a classical theorem absent from Mathlib. No claim of progress on RH is made or implied.
- **Goldbach.** The local covariance kernel (§6) is exact; its transfer to the global residual is a preregistered conjecture with a stated falsifier, never a theorem.
- $\mathrm{Aut}(C_5) \cong D_5$**.** The faithful action is proved; the reverse cardinality bound is open (§5).

8 Reproducibility

The entire core is public. Each theorem's exact statement, axiom footprint, and independent AXLE verdict are in `registry/theorems.json`; Table 1 is generated from it. The pinned environment is `leanprover/lean4:v4.32.0` with Mathlib `v4.32.0`. Independent verification is performed by AXLE at `lean-4.32.0`; local reproduction is `lake exe cache get && lake build` followed by regenerating the registry.

Remark 8.1 (Honest limitation). At the time of writing, independent verification is carried by AXLE (a third-party cloud re-check at the exact toolchain); a from-source local `lake build` was not completed in our environment because the Mathlib cache was unreachable there, and continuous integration runs the local build where the cache is available. The registry marks the local-build leg accordingly rather than overstating it.

9 Related work

The formalisation rests on Lean 4 and Mathlib [1]. Proof generation and porting used Aristotle [2]; independent verification used AXLE [3]. The methodological contribution — registers, triple verification, and a generated registry — is in the spirit of reproducible and adversarially-audited formal mathematics, adapted to a workflow in which proofs are machine-generated and must be treated as untrusted until independently checked.

10 Conclusion

A large exploratory corpus, disciplined by a named-failure-mode ledger and a triple-verification gate, yields a compact, independently-verified core: 190 theorems on the pentagonal structure of prime residues, two of which single out $p = 5$ by spectral rigidity. The same discipline draws a bright line around what remains open. We offer both the mathematics and the method, and invite others to check every line.

A The verified registry

Table 1 is generated verbatim from `registry/theorems.json`.

Table 1: The verified core: every PROVED/CONDITIONAL declaration in the registry, its module, register, whether its `#print axioms` is clean (only `propext, Classical.choice, Quot.sound`), and its independent AXLE verdict at `lean-4.32.0`.

Theorem	Module	Register	Axioms	AXLE
`admissibility_count_five`	Admissibility	PROVED	✓	verified
`admissibility_count_three`	Admissibility	PROVED	✓	verified
`universal_admissibility_count`	Admissibility	PROVED	✓	verified
`dihedral_action_faithful`	Automorphism	PROVED	✓	verified
`mul_rr`	Automorphism	PROVED	✓	verified
`mul_rsr`	Automorphism	PROVED	✓	verified
`mul_srr`	Automorphism	PROVED	✓	verified
`mul_srsr`	Automorphism	PROVED	✓	verified
`refl_map_adj`	Automorphism	PROVED	✓	verified
`rot_map_adj`	Automorphism	PROVED	✓	verified
`ten_le_card_aut`	Automorphism	PROVED	✓	verified
`cos_2pi_5`	Connectivity	PROVED	✓	verified
`lambda2_eq`	Connectivity	PROVED	✓	verified

Theorem	Module	Register	Axioms	AXLE
one_div_phi	Connectivity	PROVED	✓	verified
pentagon_lambda2_phi	Connectivity	PROVED	✓	verified
pentagon_ratio	Connectivity	PROVED	✓	verified
two_cos_4pi_5	Connectivity	PROVED	✓	verified
binet_formula	Core	PROVED	✓	verified
cos_2pi_5	Core	PROVED	✓	verified
cos_pi_div_five_eq_phi_div_two	Core	PROVED	✓	verified
each_ray_has_infinitely_many_primes	Core	PROVED	✓	verified
fib_five_dvd	Core	PROVED	✓	verified
fifth_root_of_unity	Core	PROVED	✓	verified
one_lt_phi	Core	PROVED	✓	verified
phi_pos	Core	PROVED	✓	verified
phi_sq	Core	PROVED	✓	verified
ray_add	Core	PROVED	✓	verified
ray_mul	Core	PROVED	✓	verified
ray_ne_zero_infinite	Core	PROVED	✓	verified
ray_zero_iff_dvd	Core	PROVED	✓	verified
d5_card	Geometry	PROVED	✓	verified
golden_ratio_in_C5_spectrum	Geometry	PROVED	✓	verified
pentagon_golden_diagonal	Geometry	PROVED	✓	verified
pentagon_two_distances	Geometry	PROVED	✓	verified
gCount_centered	GoldbachComb	PROVED	✓	verified
gCount_eq	GoldbachComb	PROVED	✓	verified
local_covariance	GoldbachComb	PROVED	✓	verified
factor_ratio	GoldbachLemmas	PROVED	✓	verified
gFactor_eq_one_add	GoldbachLemmas	PROVED	✓	verified
gFactor_pos	GoldbachLemmas	PROVED	✓	verified
gFactor_pos_prime	GoldbachLemmas	PROVED	✓	verified
gProduct_le_of_subset	GoldbachLemmas	PROVED	✓	verified
gProduct_pos	GoldbachLemmas	PROVED	✓	verified
gResidues_card_ne_zero	GoldbachLemmas	PROVED	✓	verified
gResidues_card_zero	GoldbachLemmas	PROVED	✓	verified
localDensity_ne	GoldbachLemmas	PROVED	✓	verified
localDensity_zero	GoldbachLemmas	PROVED	✓	verified
one_le_gProduct	GoldbachLemmas	PROVED	✓	verified
one_lt_gFactor	GoldbachLemmas	PROVED	✓	verified
one_lt_gFactor_prime	GoldbachLemmas	PROVED	✓	verified
tFactor_eq	GoldbachLemmas	PROVED	✓	verified
tFactor_lt_one	GoldbachLemmas	PROVED	✓	verified
tFactor_pos	GoldbachLemmas	PROVED	✓	verified
three_le_of_prime_ne_two	GoldbachLemmas	PROVED	✓	verified
goldbachCount_four	GoldbachSchema	PROVED	✓	verified
goldbachCount_ten	GoldbachSchema	PROVED	✓	verified
goldbach_beyond_of_model	GoldbachSchema	CONDITIONAL	✓	verified
goldbach_from_spectral_model	GoldbachSchema	CONDITIONAL	✓	verified
hasGoldbachRep_eight	GoldbachSchema	PROVED	✓	verified

Theorem	Module	Register	Axioms	AXLE
hasGoldbachRep_four	GoldbachSchema	PROVED	✓	verified
hasGoldbachRep_of_count_pos	GoldbachSchema	PROVED	✓	verified
hasGoldbachRep_six	GoldbachSchema	PROVED	✓	verified
Gamma _ne_zero_of_nontrivial	RiemannScaffold	PROVED	✓	verified
RH_of_BrockianSystem	RiemannScaffold	CONDITIONAL	✓	verified
RiemannHypothesis_of_forall_xi_zero	RiemannScaffold	PROVED	✓	verified
riemannXi_eq_zero_of_nontrivial_zeta_zero	RiemannScaffold	PROVED	✓	verified
symmetric_eigenvalue_im_zero	RiemannScaffold	PROVED	✓	verified
H3_det	Sieve	PROVED	✓	verified
H3_ground	Sieve	PROVED	✓	verified
H3_middle	Sieve	PROVED	✓	verified
H3_top	Sieve	PROVED	✓	verified
H3_trace	Sieve	PROVED	✓	verified
compatible_closure	Sieve	PROVED	✓	verified
no_adjacent_admissible	Sieve	PROVED	✓	verified
run3_signature	Sieve	PROVED	✓	verified
run_cap	Sieve	PROVED	✓	verified
silver_gap_rigidity_finite	Sieve	PROVED	✓	verified
tau_injective_on_period	Sieve	PROVED	✓	verified
tau_minimal_period	Sieve	PROVED	✓	verified
tau_period	Sieve	PROVED	✓	verified
twin_admissible_card	Sieve	PROVED	✓	verified
twin_pins_mod_three	Sieve	PROVED	✓	verified
localFactorAt_eq	SingularSeries	PROVED	✓	verified
localFactorAt_of_not_prime	SingularSeries	PROVED	✓	verified
localFactorAt_pos	SingularSeries	PROVED	✓	verified
local_factor_asymptotic	SingularSeries	PROVED	✓	verified
local_factor_denom_ne_zero	SingularSeries	PROVED	✓	verified
local_factor_of_nu_p_eq_p	SingularSeries	PROVED	✓	verified
local_factor_of_nu_p_eq_zero	SingularSeries	PROVED	✓	verified
local_factor_pos	SingularSeries	PROVED	✓	verified
nu_p'	SingularSeries	PROVED	✓	verified
nu_p_eq_image_card	SingularSeries	PROVED	✓	verified
nu_p_lt_p_of_admissible	SingularSeries	PROVED	✓	verified
singular_series_finite_pos	SingularSeries	PROVED	✓	verified
singular_series_pos	SingularSeries	CONDITIONAL	✓	verified
cos_two_pi_div_five	Spectral	PROVED	✓	verified
golden_in_cycleSpectrum_five	Spectral	PROVED	✓	verified
golden_sub_one_eq_two_cos	Spectral	PROVED	✓	verified
golden_unique_to_five	Spectral	PROVED	✓	verified
neg_golden_in_C5_spectrum	Spectral	PROVED	✓	verified
two_cos_four_pi_div_five	Spectral	PROVED	✓	verified
abs_primeGaussian_le_two	SpectralGate1	PROVED	✓	verified
coeFn_mulLpCLM	SpectralGate1	PROVED	✓	verified
coeFn_mulLpFun	SpectralGate1	PROVED	✓	verified
coeFn_primeGaussianMulCLM	SpectralGate1	PROVED	✓	verified

Theorem	Module	Register	Axioms	AXLE
continuous_primeBump	SpectralGate1	PROVED	✓	verified
continuous_primeGaussian	SpectralGate1	PROVED	✓	verified
eLpNorm_mulLpFun_le	SpectralGate1	PROVED	✓	verified
isSelfAdjoint_mulLpCLM	SpectralGate1	PROVED	✓	verified
isSelfAdjoint_primeGaussianMulCLM	SpectralGate1	PROVED	✓	verified
primeBump_le_geom	SpectralGate1	PROVED	✓	verified
primeBump_nonneg	SpectralGate1	PROVED	✓	verified
primeGaussian_le_two	SpectralGate1	PROVED	✓	verified
primeGaussian_nonneg	SpectralGate1	PROVED	✓	verified
primeGaussian _memLp_top	SpectralGate1	PROVED	✓	verified
primeGaussian _norm_le	SpectralGate1	PROVED	✓	verified
summable_primeBump	SpectralGate1	PROVED	✓	verified
brockian_table_card	TransitionKernel	PROVED	✓	verified
cousin_pins_mod_three	TransitionKernel	PROVED	✓	verified
cousin_run_cap	TransitionKernel	PROVED	✓	verified
forbidden_transition	TransitionKernel	PROVED	✓	verified
gap10_run_cap	TransitionKernel	PROVED	✓	verified
kernel_row_sum	TransitionKernel	PROVED	✓	verified
quadruplet_pins_mod_five	TransitionKernel	PROVED	✓	verified
sexy_free_mod_three	TransitionKernel	PROVED	✓	verified
sexy_run_cap	TransitionKernel	PROVED	✓	verified
totalSum_count	TransitionKernel	PROVED	✓	verified
totalSum_eq	TransitionKernel	PROVED	✓	verified
twin_admissible_singleton	TransitionKernel	PROVED	✓	verified
twin_pins_mod_three	TransitionKernel	PROVED	✓	verified
twin_table_card	TransitionKernel	PROVED	✓	verified
green_identity_integral	Weyl	PROVED	✓	verified
lagrange_identity	Weyl	PROVED	✓	verified
wronskian_const_one_witness	Weyl	PROVED	✓	verified
wronskian_hasDerivAt	Weyl	PROVED	✓	verified
wronskian_isConst	Weyl	PROVED	✓	verified
apply_ne_I_smul	Cayley	PROVED	✓	verified
apply_ne_neg_I_smul	Cayley	PROVED	✓	verified
deficiencySpace_eq_bot_iff	Cayley	PROVED	✓	verified
essentiallySelfAdjoint_iff	Cayley	PROVED	✓	verified
mem_orthogonal_rangeSMulSub_iff	Cayley	PROVED	✓	verified
mem_rangeSMulSub	Cayley	PROVED	✓	verified
norm_add_I_smul_eq	Cayley	PROVED	✓	verified
essSelfAdjoint_of_dense_ranges	Chain	PROVED	✓	verified
adjoint_isClosed'	Closure	PROVED	✓	verified
closure_eq_self_of_isClosed	Closure	PROVED	✓	verified
inner_adjoint_left	Closure	PROVED	✓	verified
isClosed_deficiencySet	Closure	PROVED	✓	verified
mem_deficiencySet_iff_mem_deficiencySpace	Closure	PROVED	✓	verified
smulPMap_adjoint_isClosed	Closure	PROVED	✓	verified
smulPMap_dense	Closure	PROVED	✓	verified

Theorem	Module	Register	Axioms	AXLE
`smulPMap_isClosable`	Closure	PROVED	✓	verified
`symmetric_adjoint_eq`	Closure	PROVED	✓	verified
`symmetric_closure_le_adjoint`	Closure	PROVED	✓	verified
`symmetric_domain_le_adjoint_domain`	Closure	PROVED	✓	verified
`symmetric_isClosable`	Closure	PROVED	✓	verified
`symmetric_le_adjoint`	Closure	PROVED	✓	verified
`boundary_L2_identity`	Disk	PROVED	✓	verified
`circle_key`	Disk	PROVED	✓	verified
`green_identity_integral`	Disk	PROVED	✓	verified
`integral_normSq_mono`	Disk	PROVED	✓	verified
`lagrange_identity`	Disk	PROVED	✓	verified
`radius_formula`	Disk	PROVED	✓	verified
`weyl_disk_circle`	Disk	PROVED	✓	verified
`weyl_nested_circle`	Disk	PROVED	✓	verified
`weyl_radius_antitone`	Disk	PROVED	✓	verified
`wronskian_hasDerivAt`	Disk	PROVED	✓	verified
`wronskian_isConst`	Disk	PROVED	✓	verified
`clm_deficiency_eq_bot`	ESA	PROVED	✓	verified
`clm_dense`	ESA	PROVED	✓	verified
`clm_domain`	ESA	PROVED	✓	verified
`clm_essentiallySelfAdjoint`	ESA	PROVED	✓	verified
`clm_isSymmetric`	ESA	PROVED	✓	verified
`id_essentiallySelfAdjoint`	ESA	PROVED	✓	verified
`vec_eq_zero_of_inner`	ESA	PROVED	✓	verified
`const_potential_isLimitPoint`	LP	PROVED	✓	verified
`exists_sqrt_pos_re`	LP	PROVED	✓	verified
`indep_of_wronskian_ne_zero`	LP	PROVED	✓	verified
`not_integrableOn_exp_mul_Ioi`	LP	PROVED	✓	verified
`wronskian_hasDerivAt`	LP	PROVED	✓	verified
`wronskian_isConst`	LP	PROVED	✓	verified
`IsSymmetric`	Operator	PROVED	✓	verified
`eq_zero_of_apply_eq_smul`	Operator	PROVED	✓	verified
`im_eq_zero_of_apply_eq_smul`	Operator	PROVED	✓	verified
`inner_apply`	Operator	PROVED	✓	verified
`inner_self_im`	Operator	PROVED	✓	verified
`norm_sub_smul_ge`	Operator	PROVED	✓	verified
`mem_deficiencySpace_iff`	Operator	PROVED	✓	verified
`smulPMap_domain`	Operator	PROVED	✓	verified
`smulPMap_isSymmetric`	Operator	PROVED	✓	verified

References

[1] The mathlib Community, *The Lean Mathematical Library*, CPP 2020. https://github.com/leanprover-community/mathlib4.

[2] Harmonic, *Aristotle: Automated Theorem Proving for Lean*. https://aristotle.harmonic.fun.

[3] Axiom, *AXLE: A Cloud Infrastructure for Lean 4 Theorem Proving Utilities*. https://axle.axiommath.ai.